# B.TECH RESEARCH METHODOLOGY & PROJECT BLUEPRINT WORKSHEET

**Student Name / Roll No:** Shlok (UAI05BTA2317), Jiya (UAI05BTA2313), Caleb (UAI05BTA2319)

**Branch:** BTech AI ML

**Title:** Neighbourhood-aware misbehavior detection for V2X networks (VeReMi Extension benchmark)

## PART 1: PROBLEM CONCEPTION & MOTIVATION

1. **Relevance (What practical, industrial, or social pain point does this address?):**
   Connected cars broadcast their position, speed and heading several times a second, and nearby cars act on it (collision warnings, platooning). If a vehicle sends false data, for example a spoofed position or a frozen speed, others can be misled, and an approaching car can be hidden. Cryptographic signatures prove who sent a message, not that its content is true, so vehicle makers and road authorities need on-board misbehavior detection as a second line of defence. Social impact: road safety.
2. **Interest (Why are you personally invested or curious about solving this specific problem?):**
   ***[Draft - replace with your own words]*** Road safety is a concrete place where a machine-learning error has real consequences, which makes it a good test of whether a model can be trusted. We were curious whether a car can catch a lying vehicle the way a person would, by noticing that it disagrees with what everyone around it reports, and whether that still works when some of those neighbours are lying too.

## PART 2: DATA STRATEGY

3. **Data Availability (Where will you source your data? Provide repository or collection method):**
   Public benchmark **VeReMi Extension**, downloaded from Zenodo (record 20090854, CC-BY 4.0). It is a **simulated** benchmark (F2MD / VEINS / SUMO, Luxembourg traffic scenario, 19 attack types), so no data collection is needed. Main study: 14 of the 19 attack types at 14:00-16:00 (RandomSpeedOffset archive excluded as mislabeled); 109,823 train, 29,547 validation, 68,971 test windows. Extension: 18 attacks, adding the 4 Sybil attacks (88,695 test windows). Code, logs, weights: our GitHub repository.
4. **Choice of Data (What specific features/attributes will you include, and what will you filter out?):**
   **Included** (19 per sender per second): 9 claim values (position relative to the receiver, speed, heading, movement, acceleration) and 10 plausibility checks: kinematic consistency, a road map learned from honest traffic, distance to the nearest other claim, exact self-repeat, copied trail, log movement. Each sample is a receiver's view: the 16 nearest senders over 10 seconds. Labels (dataset ground truth): *message* (attacker counted only where it sent a falsified message) and *vehicle* (every window of an attacker).
   **Filtered out:** sender identity (so the model cannot memorise vehicles), the simulated sensor-noise fields, windows where the receiver's own state is missing, and senders beyond the 16 nearest.

## PART 3: METHODOLOGY & ANALYSIS

5. **Analysis of Data (Outline your ML pipeline: preprocessing, missing value handling, model selection):**
   - **Preprocessing:** parse the JSON logs, bin messages to 1-second steps, build windows every 3 seconds, compute the 19 inputs (road map from the training hour only).
   - **Missing values:** a sender not heard in a step is zero-filled and marked by a presence flag; the model masks those steps.
   - **Model selection:** rule baseline (speed check); first model = transformer without checks; final model (v2) = small transformer (144k parameters) with temporal and cross-sender (neighbour) attention, plus checks; ablation = same model without neighbour attention. Training adds careful injected lies (position offset, fake stop). Optional receiver-side memory. Weighted binary cross-entropy (AdamW); alarm threshold chosen on validation data; 2 seeds.

## PART 4: VALIDATION, GENERALIZATION, & INTERPRETATION

6. **Generalization (How will you test if your model works beyond your specific training dataset?):**
   - **Done:** train on the first hour, test on the later hour (unseen time), receiver-disjoint validation and test blocks; collusion stress test with injected consistent lies (incl. a convoy never seen in training).
   - **Open:** external check on original VeReMi / VeReMi NextGen; dense traffic for v2; a 3rd seed.
7. **Analysis Validity Testing:**
   **Null Hypothesis (H0):** the neighbour-aware detector has the same F1 as the same model without neighbour attention. Alternative (H1): it is higher.
   **Evaluation Metrics:** precision, recall, F1, accuracy, false-positive rate, detection latency (seconds to first alarm), and colluder recall at a matched 1% false-positive rate.
   **Statistical Test:** paired bootstrap 95% confidence interval on the F1 difference (over receivers), McNemar's test on paired decisions, Holm correction across the 14 attacks; 2 seeds. Result: +0.3 F1 points, CI about [0.2, 0.4], significant on 8-10 of 14 attacks, so H0 is rejected.
8. **Interpretation (What actionable engineering or real-world insight will your final results yield?):**
   - Final model (v2 + memory), message labels: precision 98.16, recall 97.78, F1 97.97, accuracy 98.74 (v2 alone: F1 97.88, FPR 0.60%). Vehicle labels: v2 + memory recall 96.09, F1 97.12. First model F1 94.9; speed-check rule F1 78.9. Mean 0.9 s to first alarm; 96% of attacker-receiver pairs flagged.
   - Sybil extension (18 attacks), message labels: P 96.30, R 95.46, F1 95.88, accuracy 97.04, FPR 2.07%; vehicle F1 95.38. Neighbour attention about +1.3 F1; main miss: GridSybil ghosts (recall 0.82).
   - Reference survey (Abdel Hakeem & Kim, T-ITS 2025): 97.37% on the original VeReMi (5 attacks, no Sybil). 14 attacks: above it on all four metrics (message labels); 18 attacks: all four below it. Not like-for-like.
   - Neighbour attention cuts false alarms (FPR 0.90% to 0.60%), but its gain turns slightly negative when attackers are most of the neighbours. Plausibility checks plus lie training raise recall on careful colluding lies from at most 0.04 to 0.70-0.90; lies that stay on the road remain hard (0.23-0.48). Insight: deploy behind message signatures; on-road lies need an independent signal (signal strength, sensors, RSUs). Limit: simulated data.